

MNIST Generative Models Lab: From Manual Classifiers to VAE, DDPM, and GANs

Technical report (portfolio project)
mnist-generative-models-lab

July 2026

Abstract

I built a notebook-driven MNIST lab with two tracks: classification (from-scratch KNN and logistic regression through a PyTorch CNN) and generation (VAE, pixel-space DDPM, DCGAN, and a latent diffusion MLP). On the saved notebook outputs I kept, test accuracy climbs from **0.9705** (GPU KNN, $k=3$) to **0.9913** (comparison CNN), with logistic regression sitting lower at **0.9241** because it is strictly linear. The generative notebooks train end-to-end and produce readable digit samples, but I did not compute FID or Inception Score; my conclusions about sample quality are qualitative, based on the PNGs exported to `results/figures/`. I document both what worked as a learning path and what I would refactor next (duplicate VAE training, misleading notebook filename, missing quantitative generative metrics).

1 Introduction

I chose MNIST [2] because it is the standard place to make model behaviour visible before scaling up. This repo is not a production system; it is a curriculum I can walk through in Jupyter: understand the data, implement classical models by hand, add a small CNN, benchmark everything side by side, then explore compression (VAE), denoising (DDPM), adversarial training (DCGAN), and finally diffusion in a 16-D latent space.

Figure 1 shows the order I used. Each notebook preserves embedded outputs so metrics and plots are inspectable without retraining. All figures in this report were copied from those outputs into `results/figures/`; I did not regenerate checkpoints from gitignored `training_results/` or `*.pth` files.

MNIST lab learning path (classification → generative)

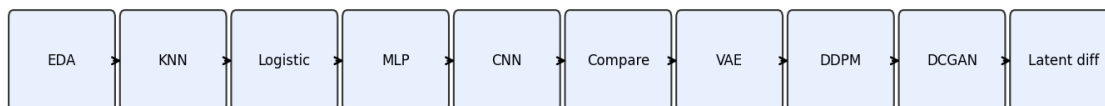


Figure 1: Learning path I followed: EDA → manual classifiers → CNN → multi-model comparison → generative models.

2 Exploratory data analysis

Before training anything, `eda_analysis.ipynb` loads MNIST and walks through class balance, per-digit galleries, mean templates, pixel variance maps, intensity histograms, and PCA structure. On visual inspection, the class counts are nearly even (Figure 2a), which means raw accuracy is a reasonable metric and I do not need heavy rebalancing tricks for this dataset.

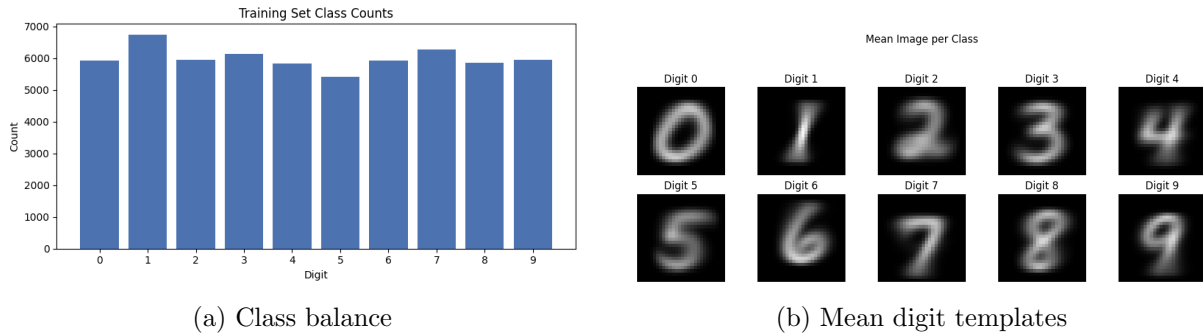


Figure 2: EDA findings from my saved notebook run. Digits are centred and stroke thickness varies by class.

The mean images (Figure 2b) look like the digits I expect, but PCA plots (Figure 3) show substantial overlap between classes in the first two components. That overlap is my main motivation for non-linear models: a single linear boundary in pixel space cannot separate all ten digits cleanly, even though MNIST is “easy” by modern standards.

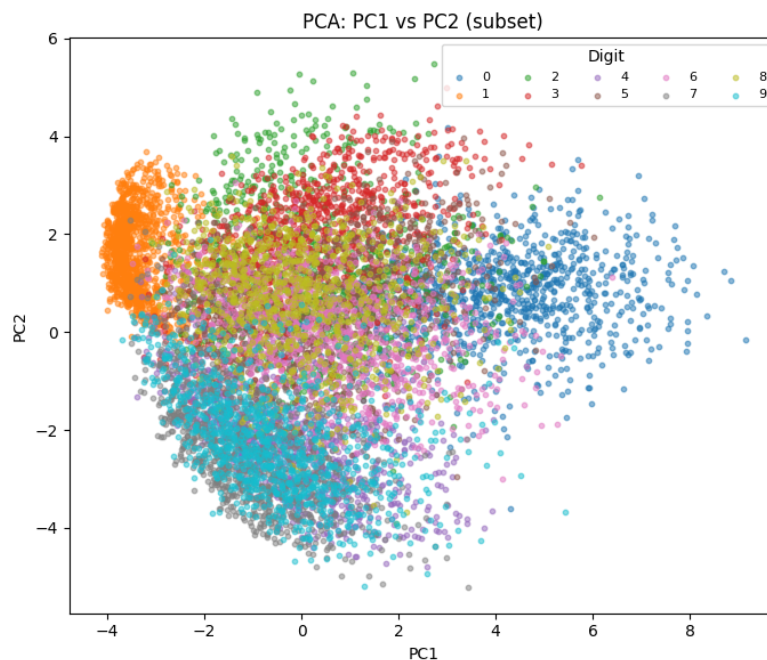


Figure 3: PCA structure in the training set. Classes overlap in low-dimensional projections, which foreshadows linear-model limits.

3 Classical and manual machine learning

3.1 KNN and the bias–variance tradeoff

In `manual_knn.ipynb` I implemented KNN in NumPy and then batched it on GPU with `torch.cdist`. A sweep over $k \in \{1, 3, 5, 9, 15, 20\}$ (Figure 4) shows accuracy peaking near $k=3$ at **0.9705** on the full test set. Very small k is noisier; very large k oversmooths decision boundaries. I found this plot more instructive than the final scalar accuracy because it makes the tradeoff concrete.

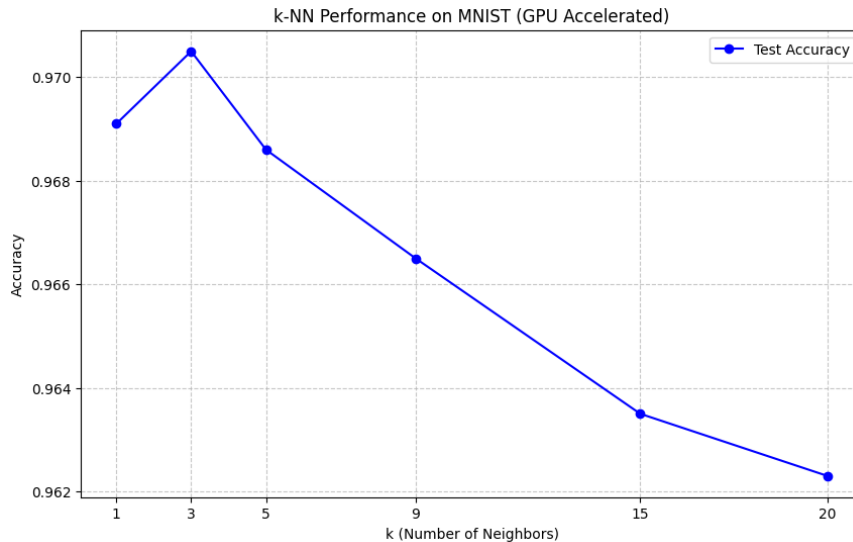


Figure 4: GPU KNN accuracy vs. k from my notebook outputs.

3.2 Logistic regression: a deliberate dip in the ladder

I expected accuracy to improve monotonically as models gained capacity. That did not happen. My from-scratch softmax regression in `manual_logistic_reg.ipynb` plateaus at **0.9241**, below KNN and well below the manual MLP. I think that is mostly a capacity issue: logistic regression is a linear classifier in pixel space, and MNIST digits written in different styles still overlap when flattened to 784 dimensions. Figure 5 shows the kind of errors I get—confusions between visually similar strokes (e.g. 4 vs 9, 3 vs 5).

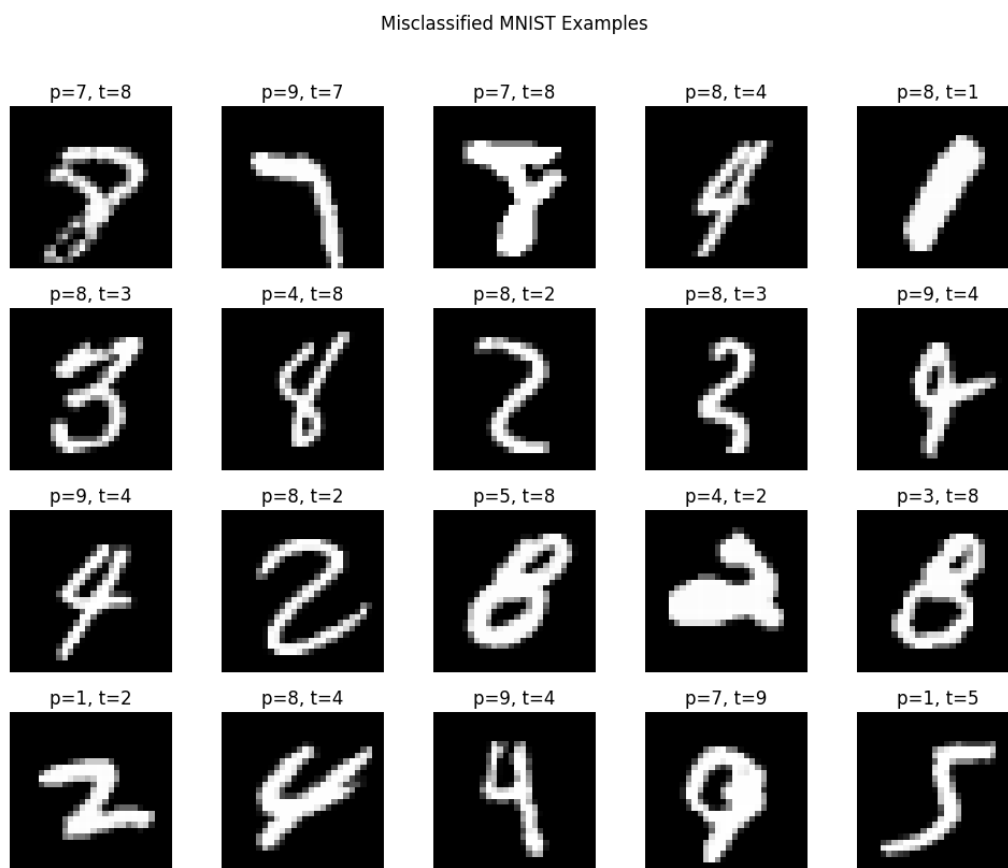


Figure 5: Misclassified test digits from my logistic regression run.

3.3 Manual MLP

Adding one ReLU hidden layer ($H=128$, 100 epochs) in `manual_mlp.ipynb` raised accuracy to **0.9794**. The first-layer weight templates (Figure 6) look like crude stroke detectors; that made backprop feel less abstract than reading equations alone.

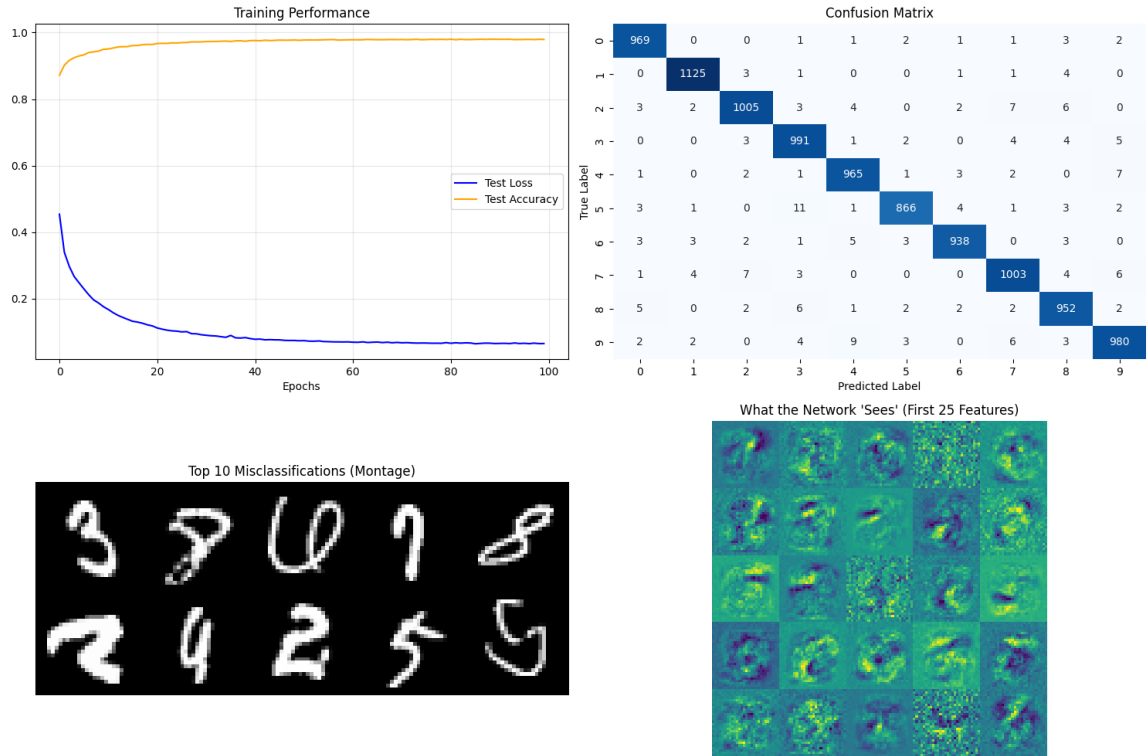


Figure 6: Manual MLP diagnostics from my run: loss/accuracy curves, confusion matrix, and first-layer filters.

Table 1 summarises the classification ladder through the manual notebooks and the standalone CNN.

Table 1: Classification metrics from saved notebook outputs (results/metrics/classification.json).

Model	Test accuracy	Source notebook
KNN (GPU, $k=3$)	0.9705	manual_knn.ipynb
Logistic regression	0.9241	manual_logistic_reg.ipynb
Manual MLP	0.9794	manual_mlp.ipynb
SimpleCNN	0.9871	cnn.ipynb
Comparison CNN	0.9913	02_mnist_models.ipynb

4 CNN baseline

cnn.ipynb implements a small SimpleCNN: two convolutional layers, max pooling, and two linear layers, trained five epochs with SGD. Test accuracy reaches **0.9871** by epoch 4. The jump from manual MLP to CNN is modest in absolute terms on MNIST, but I think the spatial inductive bias matters: convolutions reuse filters across the image instead of treating every pixel independently.

Figure 7 shows my post-training analysis panel (confusion matrix, ROC-style curves, per-class behaviour). Figure 8 visualises first-layer conv filters—mostly edge and blob detectors, which matches what I expected from coursework examples.

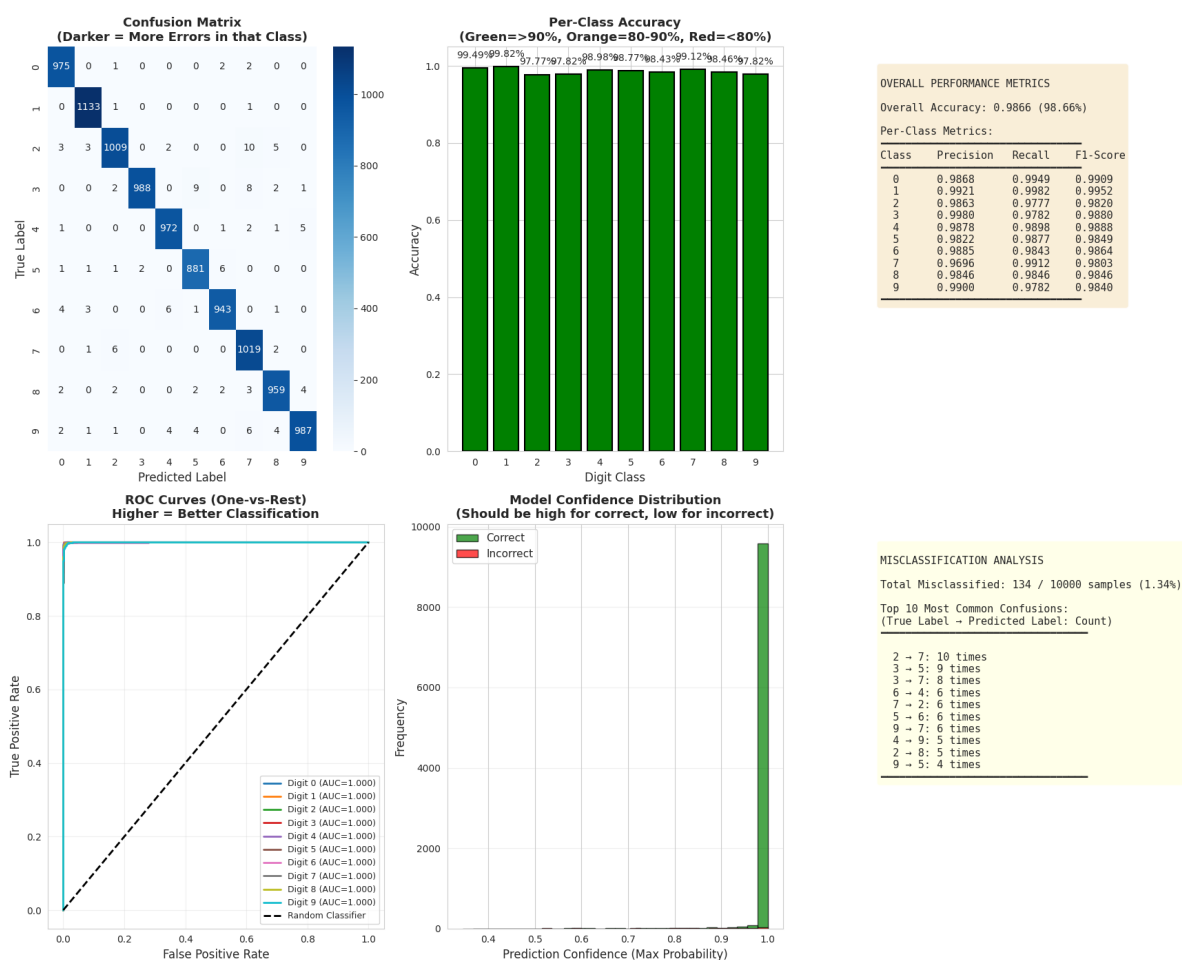


Figure 7: CNN evaluation panel from my saved run (confusion matrix and ROC-related plots in one figure).

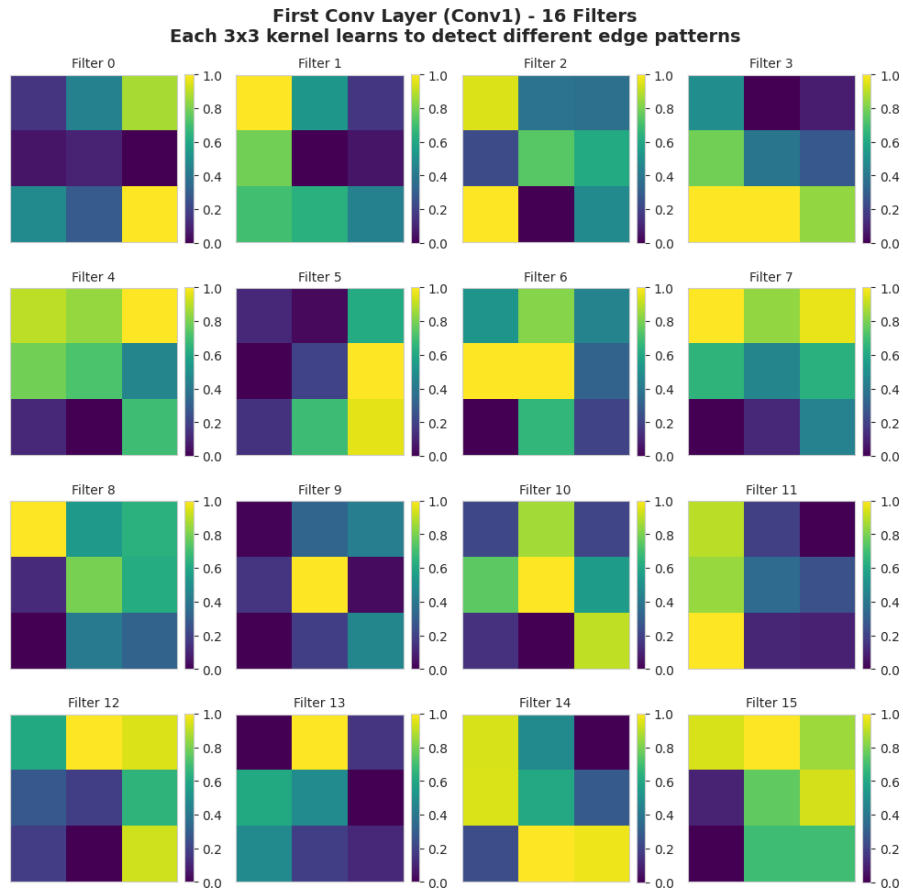


Figure 8: Learned filters in the first convolutional layer.

5 Generative models

After classification, I worked through four generative approaches in increasing complexity. I did not run FID or IS; I judged samples by eye on the PNG grids in `results/figures/`.

5.1 Variational autoencoder

`vae.ipynb` trains a convolutional VAE with a 16-D latent space for 50 epochs. Final train/validation total loss is **100.22 / 100.09** (reconstruction plus weighted KL). Reconstructions (Figure 9a) preserve digit identity but look soft, which is what I usually see with Bernoulli/BCE or MSE-style decoders on MNIST.

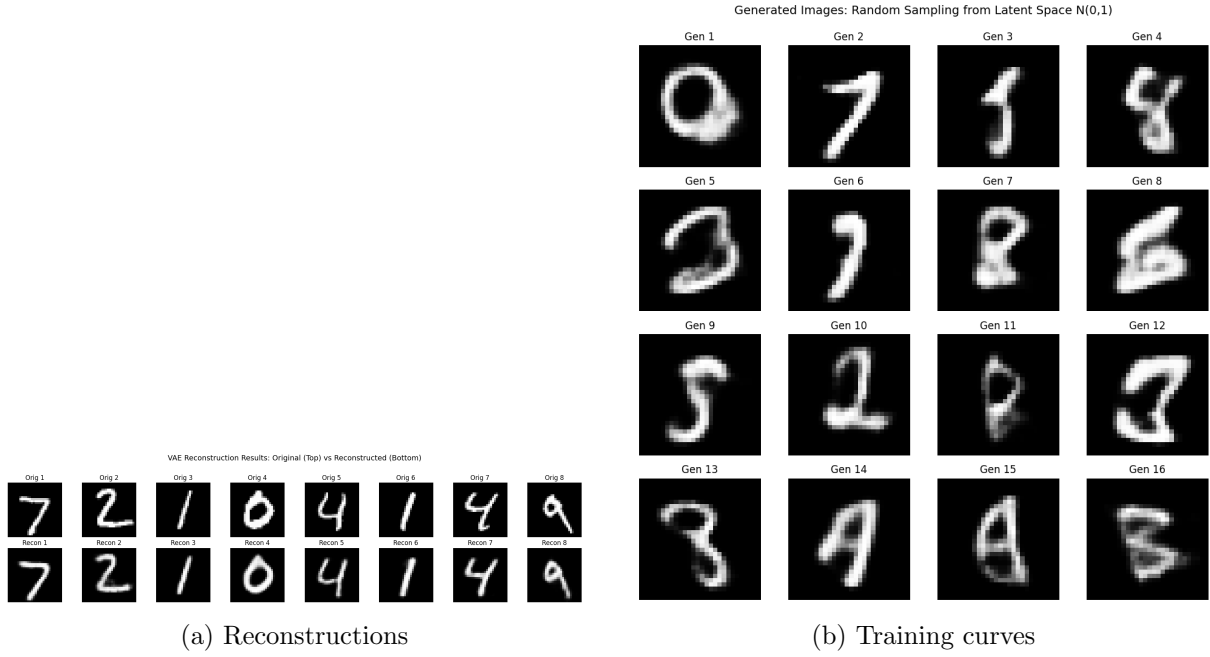


Figure 9: VAE outputs from `vae.ipynb`.

5.2 Pixel-space DDPM

`ddpm.ipynb` implements a DDPM [1] with $T=300$ timesteps and a U-Net denoiser, trained 15 epochs. Average loss at epoch 15 is **0.0454** (Figure 10a). Generated samples (Figure 10b) look like MNIST digits on inspection, though training in full 784-D pixel space felt slower and heavier than the latent diffusion stage later.

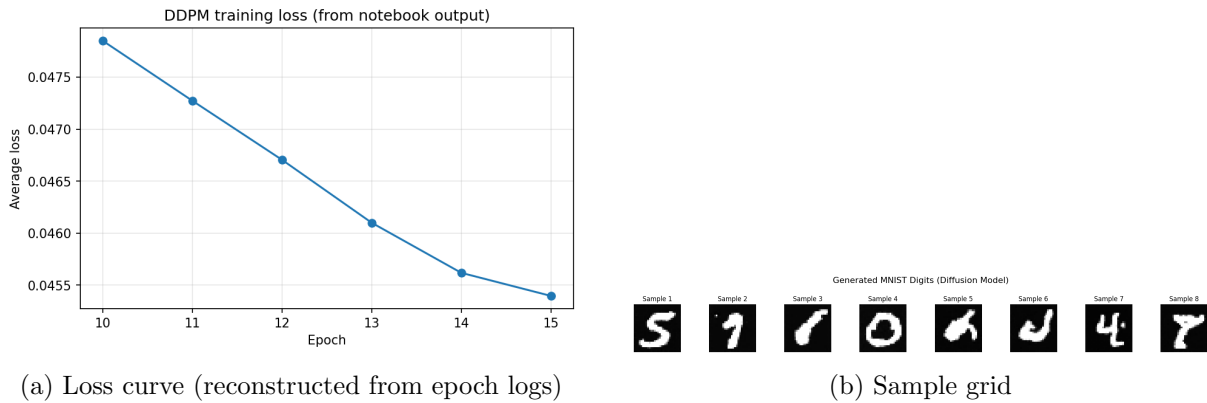


Figure 10: DDPM training and samples from my notebook outputs.

5.3 DCGAN

`drgan.ipynb` trains a DCGAN [3] for 50 epochs. Final generator/discriminator losses are **3.41** / **0.32**. The sample grid (Figure 11a) shows recognisable digits; interpolation (Figure 11b) suggests the latent space is at least locally smooth. I would not call GAN training “stable” here—the loss balance moved around—but the saved outputs are usable for demonstration.

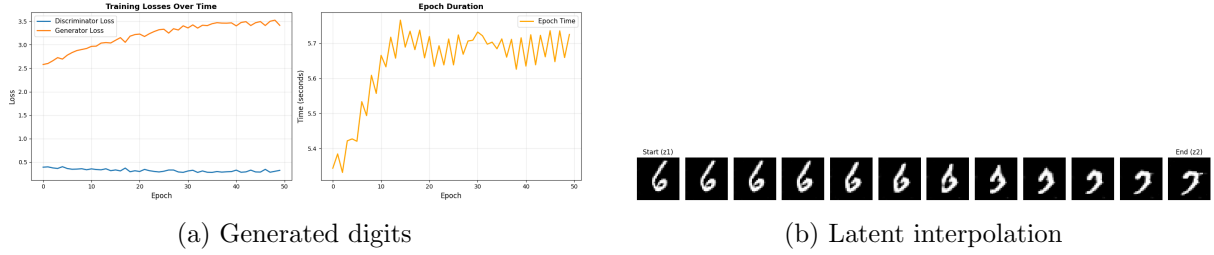


Figure 11: DCGAN outputs from my saved run.

5.4 Latent diffusion MLP

The notebook `vae + 1d unet (latent diff) (1).ipynb` retrains a VAE, extracts latents, and trains a `LatentDiffusionMLP` for 150 epochs (MSE **0.224** near the end). Despite “1d unet” in the filename, the denoiser is an MLP, not a U-Net—a naming mistake I should fix. Figure 12b shows reconstructions and generated samples from that notebook. Functionally, this is the pattern I later used on CIFAR at larger scale: compress first, diffuse second.

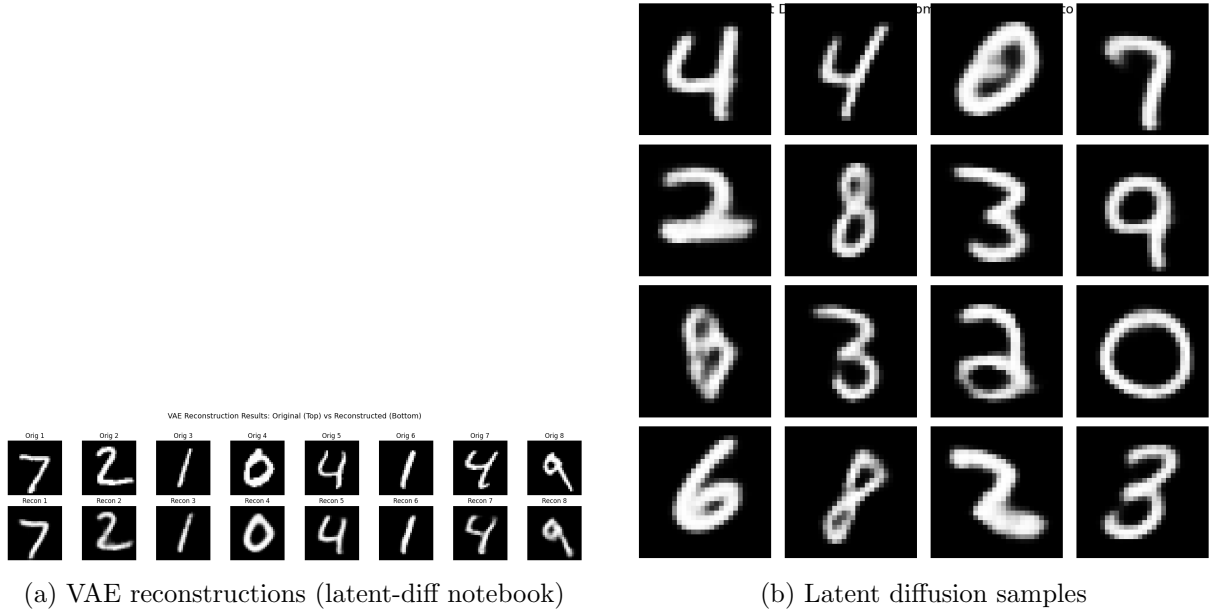


Figure 12: Latent diffusion track. Samples are digit-like on visual inspection; sharpness is limited by the VAE bottleneck.

6 Comparison and insights

`02_mnist_models.ipynb` benchmarks dummy, sklearn, and PyTorch models on a shared harness. The sorted accuracy bar chart (Figure 13a) makes the ladder obvious: deep learning wins on MNIST, but classical KNN is already strong, and the gap from MLP to CNN is incremental rather than dramatic. The comparison CNN reaches **0.9913**; remaining errors cluster on visually similar pairs (Figure 13b).

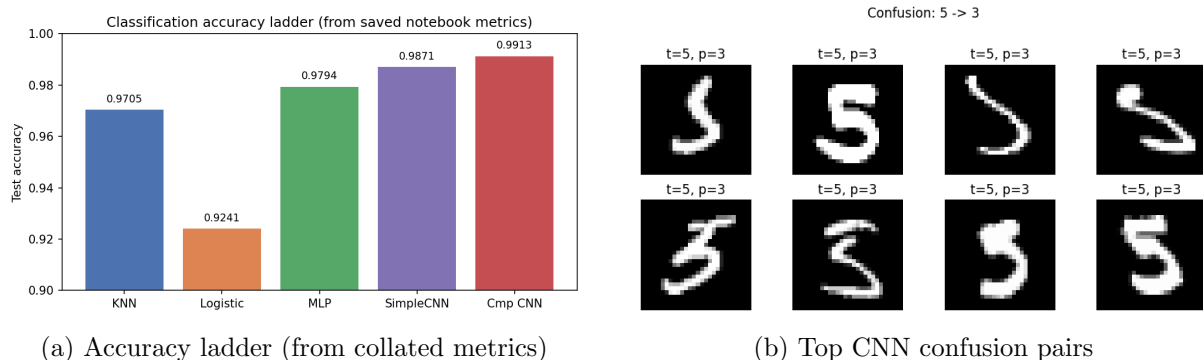


Figure 13: Multi-model comparison insights from my saved outputs.

On the generative side, I think of the tradeoffs this way: VAEs give cheap latent codes but blurry decoders; pixel DDPMs are faithful to the DDPM math but expensive in 784-D; GANs can look sharp but need careful monitoring; latent diffusion is efficient once latents exist but inherits VAE quality. I did not run controlled ablations to rank them quantitatively.

7 Limitations

- **No FID / IS.** I did not compute standard generative metrics. Claims about sample quality are qualitative.
- **28×28 grayscale only.** Findings may not transfer to colour or high-resolution data.
- **Duplicate VAE training.** `vae.ipynb` and the latent-diffusion notebook both train a VAE; I should load a checkpoint instead of retraining.
- **Misleading filename.** The latent-diffusion notebook name references a 1D U-Net; the implementation is an MLP.
- **Notebook hygiene.** `02_mnist_models.ipynb` may list duplicate CNN rows from re-runs; I cite **0.9913** as the comparison result.
- **Reproducibility.** `requirements.txt` does not pin versions; I relied on embedded outputs as the documentation source of truth.

8 Future work

- Deduplicate the VAE pipeline (train once, load `vae_model.pth` elsewhere).
- Add FID or a simple classifier-based generative score.
- Rename the latent-diffusion notebook and guard Colab-only cells for local runs.
- Extend the same notebook structure to Fashion-MNIST.
- Pin `torch` and `tensorflow` versions and add a lightweight `nbconvert` smoke test in CI.

9 Conclusion

I built an end-to-end MNIST lab that reads like a course project I actually ran: EDA, manual implementations, a CNN benchmark, and four generative families with saved plots in `results/`. Classification reaches **0.9913** on the comparison CNN; generative models produce inspectable digit samples without me claiming state-of-the-art numbers. I think documenting the logistic-regression dip, the soft VAE reconstructions, and the missing FID metrics is as important as listing the peak accuracies—it makes the repo credible for someone reviewing my work.

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.
- [2] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [3] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2016.